



OPERATING SYSTEM CONCEPTS

Chapter 6. Process Synchronization

A/Prof. Kai Dong

dk@seu.edu.cn

School of Computer Science and Engineering,
Southeast University

April 11, 2024



Contents

1 Condition Variables

2 Semaphores

3 Monitors



Condition Variables

Implementing `pthread_join()`

- Two problems:

```
1 void *child(void *arg) {  
2     printf("child\n");  
3     // Problem #1: how to indicate we are done?  
4     return NULL;  
5 }  
6  
7 int main(int argc, char *argv[]) {  
8     printf("parent:-begin\n");  
9     pthread_t c;  
10    pthread_create(&c, NULL, child, NULL);  
11    // Problem #2: how to wait for child?  
12    printf("parent:-end\n");  
13    return 0;  
14 }
```



Condition Variables

Implementing `pthread_join()` (contd.)

- Using a shared variable:

```
1 void *child(void *arg) {  
2     printf("child\n");  
3     done = 1;  
4     return NULL;  
5 }  
6  
7 int main(int argc, char *argv[]) {  
8     printf("parent:-begin\n");  
9     pthread_t c;  
10    pthread_create(&c, NULL, child, NULL);  
11    while (done == 0)  
12        ;  
13    printf("parent:-end\n");  
14    return 0;  
15 }
```

- Correct, but
- Waste of CPU time — spin-waiting.
- Do we need a lock here? Not a critical section.



Condition Variables

Implementing `pthread_join()` (contd.)

- A condition variable is an explicit queue
- The POSIX calls:
 - `wait()`: put itself to sleep;
 - `signal()`: wake a sleeping thread waiting on this condition.

```
1 pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);  
2 pthread_cond_signal(pthread_cond_t *c);
```



Condition Variables

Implementing `pthread_join()` (contd.)

```

1  int done = 0;
2  pthread_mutex_t m =
3      PTHREAD_MUTEX_INITIALIZER;
4  pthread_cond_t c =
5      PTHREAD_COND_INITIALIZER;
6
7  void thr_exit() {
8      pthread_mutex_lock(&m);
9      done = 1;
10     pthread_cond_signal(&c);
11     pthread_mutex_unlock(&m);
12 }
13
14 void *child(void *arg) {
15     printf("child\n");
16     thr_exit();
17     return NULL;
18 }

```

```

1  void thr_join() {
2      pthread_mutex_lock(&m);
3      while (done == 0)
4          pthread_cond_wait(&c, &m);
5      pthread_mutex_unlock(&m);
6  }
7
8  int main(int argc, char *argv[]) {
9      printf("parent:-begin\n");
10     pthread_t p;
11     pthread_create(&p, NULL, child, NULL);
12     thr_join();
13     printf("parent:-end\n");
14     return 0;
15 }

```



Condition Variables

Implementing `pthread_join()` (contd.)

- **Condition variable** is always with a **lock** and a **flag**
- Why are the following codes broken?

```

1 void thr_exit() {
2     pthread_mutex_lock(&m);
3     pthread_cond_signal(&c);
4     pthread_mutex_unlock(&m);
5 }
6
7 void thr_join() {
8     pthread_mutex_lock(&m);
9     pthread_cond_wait(&c, &m);
10    pthread_mutex_unlock(&m);
11 }
12
13 /* broken code #1 */
  
```

```

1 void thr_exit() {
2     done = 1;
3     pthread_cond_signal(&c);
4 }
5
6
7 void thr_join() {
8     if (done == 0)
9         pthread_cond_wait(&c);
10 }
11
12
13 /* broken code #2 */
  
```



Condition Variables

The Producer-Consumer Problem

```
1  /* Let's begin with only 1 producer, 1 consumer, buffer.size = 1 */
2
3  cond_t cond;
4  mutex_t mutex
5  void *producer(void *arg) {
6      for (int i = 0; i < loops; i++) {
7          pthread_mutex_lock(&mutex);
8          if (count == 1)
9              pthread_cond_wait(&cond, &mutex);
10         put();           // produce
11         pthread_cond_signal(&cond);
12         pthread_mutex_unlock(&mutex);
13     }
14 }
15 void *consumer(void *arg) {
16     for (int i = 0; i < loops; i++) {
17         pthread_mutex_lock(&mutex);
18         if (count == 0)
19             pthread_cond_wait(&cond, &mutex);
20         get();           // consume
21         pthread_cond_signal(&cond);
22         pthread_mutex_unlock(&mutex);
23     }
24 }
```




Condition Variables

The Producer-Consumer Problem (contd.)

- Problematic with more threads

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
<i>lock</i>	Running		Ready		Ready	0	
<i>wait</i>	Waiting		Ready		Ready	0	Nothing to get
	Waiting		Ready	<i>lock</i>	Running	0	
	Waiting		Ready	<i>put</i>	Running	1	Buffer now full
	Ready		Ready	<i>signal</i>	Running	1	T_{c1} awoken
	Ready		Ready	<i>unlock</i>	Running	1	
	Ready		Ready	<i>lock</i>	Running	1	
	Ready		Ready	<i>wait</i>	Waiting	1	Buffer full
	Ready	<i>lock</i>	Running		Waiting	1	T_{c2} sneaks in
	Ready	<i>get</i>	Running		Waiting	0	T_{c2} grabs data
	Ready	<i>signal</i>	Running		Ready	0	T_p awoken
	Ready	<i>unlock</i>	Running		Ready	0	
<i>get</i>	Running		Ready		Ready	0	No data!



Condition Variables

The Producer-Consumer Problem (contd.)

```

1  /* Still broken */
2
3  cond_t cond;
4  mutex_t mutex
5  void *producer(void *arg) {
6      for (int i = 0; i < loops; i++) {
7          pthread_mutex_lock(&mutex);
8          while (count == 1)          // use "while" instead of "if"
9              pthread_cond_wait(&cond, &mutex);
10             put();                  // produce
11             pthread_cond_signal(&cond);
12             pthread_mutex_unlock(&mutex);
13         }
14     }
15     void *consumer(void *arg) {
16         for (int i = 0; i < loops; i++) {
17             pthread_mutex_lock(&mutex);
18             while (count == 0)        // use "while" instead of "if"
19                 pthread_cond_wait(&cond, &mutex);
20             get();                   // consume
21             pthread_cond_signal(&cond);
22             pthread_mutex_unlock(&mutex);
23         }
24     }

```



Condition Variables

The Producer-Consumer Problem (contd.)

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
<i>lock</i>	Running		Ready		Ready	0	
<i>wait</i>	Waiting		Ready		Ready	0	Nothing to get
	Waiting	<i>lock</i>	Running		Ready	0	
	Waiting	<i>wait</i>	Waiting		Ready	0	Nothing to get
	Waiting		Waiting	<i>lock</i>	Running	0	
	Waiting		Waiting	<i>put</i>	Running	1	Buffer now full
	Ready		Waiting	<i>signal</i>	Running	1	T_{c1} awoken
	Ready		Waiting	<i>unlock</i>	Running	1	
	Ready		Waiting	<i>lock</i>	Running	1	
	Ready		Waiting	<i>wait</i>	Waiting	1	Buffer full
<i>while</i>	Running		Waiting		Waiting	1	Recheck condition
<i>get</i>	Running		Waiting		Waiting	0	T_{c1} grabs data
<i>signal</i>	Running		Ready		Waiting	0	Oops, T_{c2} awoken
<i>unlock</i>	Running		Ready		Waiting	0	
<i>lock</i>	Running		Ready		Waiting	0	
<i>wait</i>	Waiting		Ready		Waiting	0	Nothing to get
	Waiting	<i>while</i>	Ready		Waiting	0	Recheck condition
	Waiting	<i>wait</i>	Waiting		Waiting	0	Everyone waiting!



Condition Variables

The Producer-Consumer Problem (contd.)

```

1  /* buffer_size = 1 */
2
3  cond_t empty, fill;           // two condition variables
4  mutex_t mutex
5  void *producer(void *arg) {
6      for (int i = 0; i < loops; i++) {
7          pthread_mutex_lock(&mutex);
8          while (count == 1)
9              pthread_cond_wait(&empty, &mutex);
10         put();                 // produce
11         pthread_cond_signal(&fill);
12         pthread_mutex_unlock(&mutex);
13     }
14 }
15 void *consumer(void *arg) {
16     for (int i = 0; i < loops; i++) {
17         pthread_mutex_lock(&mutex);
18         while (count == 0)
19             pthread_cond_wait(&fill, &mutex);
20         get();                 // consume
21         pthread_cond_signal(&empty);
22         pthread_mutex_unlock(&mutex);
23     }
24 }
  
```



Condition Variables

The Producer-Consumer Problem (contd.)

```

1  /* dealing with bounded buffer instead of single buffer */
2
3  cond_t empty, fill;           // still two condition variables
4  mutex_t mutex
5  void *producer(void *arg) {
6      for (int i = 0; i < loops; i++) {
7          pthread_mutex_lock(&mutex);
8          while (count == MAX)    // remember: always use "while"
9              pthread_cond_wait(&empty, &mutex);
10             put();              // produce
11             pthread_cond_signal(&fill);
12             pthread_mutex_unlock(&mutex);
13         }
14     }
15     void *consumer(void *arg) {
16         for (int i = 0; i < loops; i++) {
17             pthread_mutex_lock(&mutex);
18             while (count == 0)
19                 pthread_cond_wait(&fill, &mutex);
20             get();              // consume
21             pthread_cond_signal(&empty);
22             pthread_mutex_unlock(&mutex);
23         }
24     }

```



Condition Variables

Covering Condition

- Consider a memory allocation scenario:
 - Assume there are 0 bytes free;
 - Thread T_a calls *allocate*(100)
 - Thread T_b calls *allocate*(10)
 - Thread T_c calls *free*(50)
- Which thread to signal?
 - What if T_a is awoken?
- *pthread_cond_broadcast*(), to wake up all waiting threads.



Contents

1 Condition Variables

2 Semaphores

3 Monitors



Semaphores

- A semaphore is an object with an integer value that we can manipulate with two routines: *wait()* and *signal()*.
 - In the POSIX standard, these routines are *sem_wait()* and *sem_post()*.
 - Historically, Dijkstra uses *P()* and *V()*. *P()* comes from “prolaag”, a contraction of “probeer” (Dutch for “try”) and “verlaag” (“decrease”); *V()* comes from the Dutch word “verhoog” which means “increase”.

```

1 // Correct!
2 wait(s) {
3     while (s <= 0) ; // Spin!
4     s --;
5 }
6
7 signal(s) {
8     s ++;
9 }
  
```

```

1 // Incorrect!
2 wait(s) {
3     s --;
4     while (s < 0) ;
5 }
6
7 signal(s) {
8     s ++;
9 }
  
```




Semaphores

Semaphore Implementation

- To overcome the need for busy waiting:

```
1  typedef struct {
2      int value;
3      struct process *list;
4  } semaphore;
5
6  wait(semaphore *s) {
7      s->value --;
8      if (s->value < 0) {
9          add this process to s->list;
10         block();
11     }
12 }
13
14 signal(semaphore *s) {
15     s->value ++;
16     if (s->value <= 0) {
17         remove a process P from s->list;
18         wakeup(P);
19     }
20 }
```



Semaphores

Why Semaphores

$$\text{Semaphore} = \text{Mutex} + \text{Integer} + \text{ConditionVariable}$$

- Functionality
 - A single primitive for all things related to synchronization
 - Both locks and condition variables
- Correctness and Convenience
 - Avoid errors — can *signal()* first then *wait()*.
 - Clean and organized — making it easy to demonstrate their correctness.
 - Efficient.



Semaphores

Basic Synchronization Patterns

- Signaling
- Rendezvous
- Mutex
- Multiplex
- Barrier
- Reusable barrier
- Pairing



Semaphores

Signaling

```
1  /* Thread A */  
2  statement a;
```

```
1  /* Thread B */  
2  statement b;
```

- How to guarantee that a happens before b?

```
1  /* Thread A */  
2  statement a;  
3  signal(s);
```

```
1  /* Thread B */  
2  wait(s);  
3  statement b;
```

- What should semaphore *s* initially be?



Semaphores

Rendezvous

```

1  /* Thread A */
2  statement a1;
3  statement a2;

```

```

1  /* Thread B */
2  statement b1;
3  statement b2;

```

- How to guarantee that a1 happens before b2, and b1 before a2?

```

1  /* Thread A */
2  statement a1;
3  signal(a);
4  wait(b);
5  statement a2;

```

```

1  /* Thread B */
2  statement b1;
3  signal(b);
4  wait(a);
5  statement b2;

```

- What should semaphores a , b initially be?



Semaphores

Rendezvous (contd.)

- Is it correct? — Yes, but probably less efficient.

```

1  /* Thread A */
2  statement a1;
3  signal(a);
4  wait(b);
5  statement a2;
  
```

```

1  /* Thread B */
2  statement b1;
3  wait(a);
4  signal(b);
5  statement b2;
  
```

- Is it correct? — No, deadlock.

```

1  /* Thread A */
2  statement a1;
3  wait(b);
4  signal(a);
5  statement a2;
  
```

```

1  /* Thread B */
2  statement b1;
3  wait(a);
4  signal(b);
5  statement b2;
  
```



Semaphores

Mutex

```
1  /* Thread A */  
2  count = count + 1;
```

```
1  /* Thread B */  
2  count = count + 1;
```

- Add semaphores to the following example to enforce mutual exclusion to the shared variable *count*.

```
1  /* Thread A */  
2  wait(mutex);  
3  count = count + 1;  
4  signal(mutex);
```

```
1  /* Thread B */  
2  wait(mutex);  
3  count = count + 1;  
4  signal(mutex);
```

- What should semaphores *mutex* initially be?



Semaphores

Multiplex

- Generalize the previous solution so that it allows multiple threads to run in the critical section at the same time, but it enforces an upper limit (MAX) on the number of concurrent threads.

```
1 wait(multiplex);  
2 count = count + 1;    // critical section  
3 signal(multiplex);
```

- What should semaphores *multiplex* initially be?



Semaphores

Barrier

- Generalize the rendezvous solution. n threads should run the following code:

```
1 rendezvous;
2 critical point;
```

- How to ensure that no thread executes **critical point** until after all threads have executed **rendezvous**?

```
1 /* initialization */
2 int count = 0;
3 semaphore mutex = 1;
4 semaphore barrier = 0;
```

```
1 rendezvous;
2 wait(mutex);
3 count = count + 1;
4 signal(mutex);
5 if (count == n)
6     signal(barrier);
7 wait(barrier);
8 signal(barrier);
9 critical point;
```



Semaphores

Barrier (contd.)

```
1  /* Bad barrier solution */
2  rendezvous;
3  wait(mutex);
4  count = count + 1;
5  if (count == n)
6      signal(barrier);
7  wait(barrier);
8  signal(barrier);
9  signal(mutex);
10 critical point;
```

- Common source of deadlocks: blocking on a semaphore while holding a mutex.



Semaphores

Reusable Barrier (contd.)

```
1 semaphore barrier1 = 0, barrier2 = 0, mutex = 1;
2
3 rendezvous;
4 wait(mutex);
5 count += 1;
6 if (count == n)
7     for (int i = 0; i < n; i++)
8         signal(barrier1);
9 signal(mutex);
10 wait(barrier1);
11 critical point;
12 wait(mutex);
13 count -= 1;
14 if (count == 0)
15     for (int i = 0; i < n; i++)
16         signal(barrier2);
17 signal(mutex);
18 wait(barrier2);
```



Semaphores

Pairing

- Imagine that threads represent ballroom dancers and that two kinds of dancers, leaders and followers, wait in two queues before entering the dance floor. When a leader arrives, it checks to see if there is a follower waiting. If so, they can both proceed. Otherwise it waits. Similarly, when a follower arrives, it checks for a leader and either proceeds or waits, accordingly.

```
1 semaphore leader = 0, follower = 0;
```

```
1 /* leader */
2 signal(follower);
3 wait(follower);
4 dance();
```

```
1 /* follower */
2 signal(follower);
3 wait(leader);
4 dance();
```

- How to ensure *dance()* are executed in pairs?



Semaphores

Pairing (contd.)

```

1  /* initialization */
2  int num_l = 0, num_f = 0;
3  semaphore leader = 0, follower = 0, pairing = 0, mutex = 1;

```

```

1  /* leader */
2  wait(mutex);
3  if (num_f > 0) {
4      num_f--;
5      signal(leader);
6  }
7  else {
8      num_l++;
9      signal(mutex);
10     wait(follower);
11 }
12 dance();
13 wait(pairing);
14 signal(mutex);

```

```

1  /* follower */
2  wait(mutex);
3  if (num_l > 0) {
4      num_l--;
5      signal(follower);
6  }
7  else {
8      num_f++;
9      signal(mutex);
10     wait(leader);
11 }
12 dance();
13 signal(pairing);
14 // no signal(mutex);

```



Semaphore

The Producer-Consumer Problem

```
1 semaphore empty = MAX, full = 0, mutex = 1;
2
3 void *producer(void *args) {
4     for (int i = 0; i < loops; i++) {
5         wait(empty);
6         wait(mutex);
7         put();
8         signal(mutex);
9         signal(full);
10    }
11 }
12
13 void *consumer(void *args) {
14     for (int i = 0; i < loops; i++) {
15         wait(full);
16         wait(mutex);
17         get();
18         signal(mutex);
19         signal(empty);
20    }
21 }
```



Semaphore

The Dining Philosophers

- There are five “philosophers” sitting around a table. Between each pair of philosophers is a single chopstick (and thus, five total). The philosophers each have times where they think, and don’t need any chopsticks, and times where they eat. In order to eat, a philosopher needs two chopsticks, both the one on their left and the one on their right. The basic loop of each philosopher is as follows, assuming each has a unique identifier $p \in [0, 4]$:

```
1 while (true) {  
2     think();  
3     getchopsticks();  
4     eat();  
5     putchopsticks();  
6 }
```



Semaphore

The Dining Philosophers (contd.)

```
1  /* a failed solution , why failed? */
2
3  int left(int p) { return p; }
4  int right(int p) {return (p + 1) % 5; }
5
6  void putchopsticks() {
7      signal(chopsticks[ left(p)]);
8      signal(chopsticks[ right(p)]);
9  }
10
11 void getchopsticks() {
12     wait(chopsticks[ left(p)]);
13     wait(chopsticks[ right(p)]);
14 }
```




Semaphore

The Dining Philosophers (contd.)

```
1  /* a correct solution */
2
3  int left(int p) { return p; }
4  int right(int p) {return (p + 1) % 5; }
5
6  void putchopsticks() {
7      signal(chopsticks[ left(p)]);
8      signal(chopsticks[ right(p)]);
9  }
10
11 void getchopsticks() {
12     if (p == 4) {
13         wait(chopsticks[ right(p)]);
14         wait(chopsticks[ left(p)]);
15     }
16     else {
17         wait(chopsticks[ left(p)]);
18         wait(chopsticks[ right(p)]);
19     }
20 }
```



Semaphore

The Readers-Writers Problem

- Imagine a number of concurrent operations, including reads and writes.
 - Writes change the state of the data
 - Reads do not — Many reads can proceed concurrently, as long as we can guarantee that no write is on-going.
- The first readers-writers problem — requires that, no reader be kept waiting unless a writer has already obtained permission to use the shared object.
- The second readers-writers problem — requires that, once a writer is ready, that writer perform its write as soon as possible.



Semaphore

The First Readers-Writers Problem

```
1 semaphore write_mutex = 1;
2 semaphore readcount_mutex = 1;
3 int read_count = 0;
```

```
1 void write() {
2     do {
3         wait(write_mutex);
4         /* writing */
5         signal(write_mutex);
6     }
7     while (true);
8 }
```

```
1 void read() {
2     do {
3         wait(readcount_mutex);
4         read_count ++;
5         if (read_count == 1)
6             wait(write_mutex);
7         signal(readcount_mutex);
8         /* reading */
9         wait(readcount_mutex);
10        read_count --;
11        if (read_count == 0)
12            signal(write_mutex);
13        signal(readcount_mutex);
14    }
15    while (true);
16 }
```



Semaphore

The No-starve Readers-Writers Problem

```

1 semaphore readcount_mutex= 1;
2 semaphore writemutex=1;
3 int read_count = 0;
4 semaphore readmutex =1;

```

```

1 void write() {
2     do {
3         wait(readmutex);
4         wait(writemutex);
5         /* writing */
6         signal(writemutex);
7         signal(readmutex);
8     }
9     while (true);
10 }

```

```

1 void read() {
2     do {
3         wait(readmutex);
4         signal(readmutex);
5         wait(readcount_mutex);
6         read_count ++;
7         if (read_count == 1)
8             wait(writemutex);
9         signal(readcount_mutex);
10        /* reading */
11        wait(readcount_mutex);
12        read_count --;
13        if (read_count == 0)
14            signal(writemutex);
15        signal(readcount_mutex);
16    }
17    while (true);
18 }

```



Semaphore

The Second Readers-Writers Problem

```

1 semaphore readcount_mutex= 1;
2 semaphore writemutex=1;
3 int write_count = read_count = 0;
4 semaphore writecount_mutex= 1;
5 semaphore readmutex=1;

```

```

1 void write() {
2     do {
3         wait(writecount_mutex);
4         write_count ++;
5         if (write_count == 1)
6             wait(readmutex);
7         signal(writecount_mutex);
8         wait(writemutex);
9         /* writing */
10        signal(writemutex);
11        wait(writecount_mutex);
12        write_count --;
13        if (write_count == 0)
14            signal(readmutex);
15        signal(writecount_mutex);
16    }
17    while (true);
18 }

```

```

1 void read() {
2     do {
3         wait(readmutex);
4         wait(readcount_mutex);
5         read_count ++;
6         if (read_count == 1)
7             wait(writemutex);
8         signal(readcount_mutex);
9         signal(readmutex);
10        /* reading */
11        wait(readcount_mutex);
12        read_count --;
13        if (read_count == 0)
14            signal(writemutex);
15        signal(readcount_mutex);
16    }
17    while (true);
18 }

```



Semaphores

Why and Why Not Semaphores

- Why semaphores?
 - Avoid errors/Clean and organized/Efficient.
 - Synchronization between Processes.
 - How about performance? Better than you can imagine.
- Why not semaphores?
 - Signaling all (broadcast) in CV.
 - Priority inheritance in mutex locks.



Contents

1 Condition Variables

2 Semaphores

3 **Monitors**



Monitors

- Why **monitors**?
 - As **object-oriented programming** was gaining ground, people started to think about ways to merge synchronization into a more structured programming environment.
- With a monitor class — the monitor guarantees that only one thread can be active within the monitor at a time.
- A Java Monitor — add the keyword `synchronized` to the method or set of methods that you wish to use as a monitor

```
1 public class SynchronizaedCounter {  
2     private int c = 0;  
3     public synchronized void increment() {  
4         c ++;  
5     }  
6     public synchronized void decrement() {  
7         c --;  
8     }  
9     public synchronized int value() {  
10        return c;  
11    }  
12 }
```




Monitors

Hoare Vs. Mesa

- Hoare Semantics:
- The *signal()* immediately wakes one waiting thread.
- Guess which one is more popular?
- Mesa semantics:
- The *signal()* move a single waiting thread to ready state.



Monitors

Hoare Vs. Mesa (contd.)

```

1  /* correct with Hoare semantic, but incorrect with Mesa semantic */
2  monitor class BoundedBuffer {
3  private: int buffer[MAX];
4           int fill, use, fullEntires = 0;
5           cond_t empty, full;
6  public: void produce(int element) {
7           if (fullEntires == MAX)
8             // correct with Mesa changing "if" to "while"
9             wait(&empty);
10          buffer[fill] = element;
11          fill = (fill + 1) % MAX;
12          fullEntires++;
13          signal(&full);
14        }
15        int consume() {
16          if (fullEntires == 0)
17            // correct with Mesa changing "if" to "while"
18            wait(&full);
19          int tmp = buffer[use];
20          use = (use + 1) % MAX;
21          fullEntires--;
22          signal(&empty);
23          return tmp;
24        }
25    }

```